

Francisco Acuña - 20220565

Isaac Cyrman - 20220289

Rodrigo Reyes - 20200090

Documentación

Estructura del Proyecto

El proyecto está compuesto por los siguientes archivos:

1. **build_and_run.sh**: Script de shell para limpiar, compilar, enlazar y ejecutar la aplicación en QEMU.
2. **linker.ld**: Script del enlazador que define la distribución de memoria y el punto de entrada del programa.
3. **main.c**: Archivo principal de la aplicación que contiene la lógica para recibir entradas, realizar el cálculo y mostrar los resultados.
4. **root.s**: Archivo de arranque en ensamblador que inicializa la pila y llama a la función principal.
5. **stdio.c** y **stdio.h**: Archivos que implementan funciones de comunicación UART para enviar y recibir datos.
6. **string.c** y **string.h**: Funciones para convertir cadenas a enteros y enteros a cadenas.

1. build_and_run.sh

El script **build_and_run.sh** automatiza el proceso de construcción y ejecución en QEMU. El script realiza los siguientes pasos:

- **Limpia** los archivos de construcción anteriores.
- **Ensambla** el archivo **root.s** para generar el archivo objeto **root.o**.
- **Compila** los archivos **main.c**, **stdio.c** y **string.c** para generar sus respectivos archivos objeto.
- **Enlaza** los archivos objeto utilizando **linker.ld** para producir el archivo ejecutable **calculadora.elf**.
- **Convierte** el archivo ELF a formato binario (**calculadora.bin**).
- **Ejecuta** la aplicación en QEMU con una emulación ARM.

Unset

```
echo "Cleaning up previous build files..."
rm -f root.o main.o stdio.o string.o calculadora.elf
calculadora.bin

echo "Assembling startup.s..."
arm-none-eabi-as -o root.o root.s

echo "Compiling main.c..."
arm-none-eabi-gcc -c -o main.o main.c

echo "Compiling stdio.c..."
arm-none-eabi-gcc -c -o stdio.o stdio.c

echo "Compiling string.c..."
arm-none-eabi-gcc -c -o string.o string.c

echo "Linking object files..."
arm-none-eabi-ld -T linker.ld -o calculadora.elf root.o main.o
stdio.o string.o

echo "Converting ELF to binary..."
arm-none-eabi-objcopy -O binary calculadora.elf calculadora.bin

echo "Running QEMU..."
qemu-system-arm \
-M versatilepb \
-m 128M \
-nographic \
-echr 0x14 \
-kernel calculadora.bin
```

2. linker.ld

El script del enlazador especifica la distribución de la memoria y el punto de entrada para la aplicación.

Distribución de Memoria:

- La sección `.text` comienza en la dirección `0x00010000` (la dirección inicial en la RAM).
- La sección `.data` sigue a la sección `.text` y contiene variables inicializadas.
- La sección `.bss` se usa para variables no inicializadas y se coloca después de la sección `.data`.

Punto de Entrada:

El punto de entrada está definido por `_start`, que llama a la función `reset`. La función `reset` configura el puntero de pila y luego llama a la función principal.

```
Unset
ENTRY(_start)

SECTIONS
{
    . = 0x00010000; /* Starting address in RAM */

    .text :
    {
        *(.text*)
    }

    .data : AT (ADDR(.text) + SIZEOF(.text))
    {
        *(.data*)
    }

    .bss :
    {
        __bss_start = .;
        *(.bss*)
        __bss_end = .;
    }
}
```

3. main.c

El archivo principal de la aplicación que maneja la interacción con el usuario a través de UART. Lee dos enteros del usuario, los suma y muestra el resultado.

Funcionalidad:

- Muestra un mensaje para que el usuario ingrese dos números.
- Convierte la entrada del usuario (cadena) en enteros usando `str_to_int`.
- Calcula la suma de los dos números.
- Convierte la suma de vuelta a una cadena y envía el resultado al usuario a través de UART.

Código:

```
C/C++
#include "string.h"
#include "stdio.h"

void main() {
    char input1[16];
    char input2[16];
    int num1, num2, sum;

    uart_puts("Programa: Sumar dos numeros\n");

    while (1) {
        // Pedir el primer número
        uart_puts("Ingrese el primer numero: ");
        uart_gets(input1, sizeof(input1));
        num1 = str_to_int(input1);

        // Pedir el segundo número
        uart_puts("Ingrese el segundo numero: ");
        uart_gets(input2, sizeof(input2));
        num2 = str_to_int(input2);

        // Calcular la suma
        sum = num1 + num2;
```

```

        // Mostrar el resultado
        char result[32];
        int_to_str(sum, result, sizeof(result));
        uart_puts("Suma: ");
        uart_puts(result);
        uart_puts("\n");
    }
}

```

4. root.s

Este archivo de ensamblador contiene el código de arranque que configura el puntero de pila y llama a la función principal.

Funcionalidad:

- Configura el puntero de pila (*sp*) en `0x00020000`.
- Salta a la función `main` para iniciar la ejecución del programa.
- El bucle infinito al final asegura que el programa no termine.

Código:

```

Unset
.section .text
.global _start

_start:
b reset

reset:
ldr sp, =0x00020000    @ Set up stack pointer (within RAM)
bl main                @ Call the main function
loop:
b loop                 @ Infinite loop to prevent exit

```

5. `stdio.c` y `stdio.h`

Estos archivos implementan funciones básicas de comunicación UART para enviar y recibir caracteres y cadenas.

Funciones:

- `uart_putc`: Envía un solo carácter a través de UART.
- `uart_getc`: Recibe un solo carácter de UART.
- `uart_puts`: Envía una cadena a través de UART.
- `uart_gets`: Recibe una cadena de UART (hasta `max_length` caracteres).

Código (`stdio.c`):

```
C/C++
#include "stdio.h"

#define UART0_BASE 0x101f1000
#define UART_DR    0x00
#define UART_FR    0x18
#define UART_FR_TXFF 0x20
#define UART_FR_RXFE 0x10

volatile unsigned int * const UART0 = (unsigned int *)UART0_BASE;

// Enviar un solo carácter por UART
void uart_putc(char c) {
    while (UART0[UART_FR / 4] & UART_FR_TXFF);
    UART0[UART_DR / 4] = c;
}

// Recibir un solo carácter por UART
char uart_getc() {
    while (UART0[UART_FR / 4] & UART_FR_RXFE);
    return (char)(UART0[UART_DR / 4] & 0xFF);
}

// Enviar una cadena por UART
```

```

void uart_puts(const char *s) {
    while (*s) {
        uart_putc(*s++);
    }
}

// Recibir una cadena desde UART
void uart_gets(char *buffer, int max_length) {
    int i = 0;
    char c;
    while (i < max_length - 1) { // Deja espacio para el
terminador nulo
        c = uart_getc();
        if (c == '\n' || c == '\r') {
            uart_putc('\n'); // Eco de nueva línea
            break;
        }
        uart_putc(c); // Eco del carácter ingresado
        buffer[i++] = c;
    }
    buffer[i] = '\0'; // Terminar cadena
}

```

Código (stdio.h):

```

C/C++
#ifndef STDIO_H
#define STDIO_H

void uart_putc(char c);
void uart_puts(const char *s);
char uart_getc();
void uart_gets(char *buffer, int max_length);

#endif // STDIO_H

```

6. string.c y string.h

Estos archivos proporcionan funciones de utilidad para convertir cadenas a enteros (`str_to_int`) y enteros a cadenas (`int_to_str`).

Funciones:

- `str_to_int`: Convierte una cadena a un entero, manejando signos opcionales.
- `int_to_str`: Convierte un entero a una cadena, con soporte para números negativos.

Código (`string.c`):

```
C/C++
#include "string.h"

// Convertir cadena a entero
int str_to_int(const char *s) {
    int num = 0;
    int sign = 1;

    // Manejar signo opcional
    if (*s == '-') {
        sign = -1;
        s++;
    }

    while (*s >= '0' && *s <= '9') {
        num = num * 10 + (*s - '0');
        s++;
    }

    return sign * num;
}

// Convertir entero a cadena
void int_to_str(int num, char *buffer, int buffer_size) {
    int i = 0;
    int is_negative = 0;
```



```

// Manejar cero explícitamente
if (num == 0) {
    if (buffer_size > 1) {
        buffer[i++] = '0';
        buffer[i] = '\0';
    }
    return;
}

// Manejar signo negativo
if (num < 0) {
    is_negative = 1;
    num = -num;
}

// Generar dígitos en orden inverso
while (num != 0 && i < buffer_size - 1) {
    buffer[i++] = (num % 10) + '0';
    num /= 10;
}

// Añadir signo negativo si es necesario
if (is_negative && i < buffer_size - 1) {
    buffer[i++] = '-';
}

// Terminar cadena
buffer[i] = '\0';

// Invertir cadena
for (int j = 0; j < i / 2; j++) {
    char temp = buffer[j];
    buffer[j] = buffer[i - j - 1];
    buffer[i - j - 1] = temp;
}

```

```
}
```

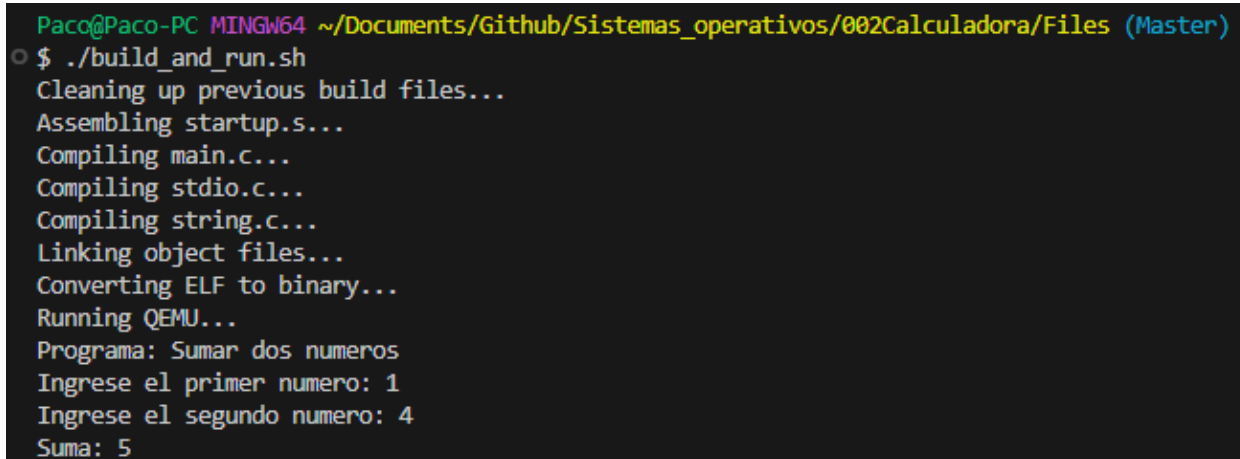
Código (string.h):

```
C/C++
#ifndef STRING_H
#define STRING_H

int str_to_int(const char *s);
void int_to_str(int num, char *buffer, int buffer_size);

#endif // STRING_H
```

Screenshot:



```
Paco@Paco-PC MINGW64 ~/Documents/Github/Sistemas_operativos/002Calculadora/Files (Master)
$ ./build_and_run.sh
Cleaning up previous build files...
Assembling startup.s...
Compiling main.c...
Compiling stdio.c...
Compiling string.c...
Linking object files...
Converting ELF to binary...
Running QEMU...
Programa: Sumar dos numeros
Ingrese el primer numero: 1
Ingrese el segundo numero: 4
Suma: 5
```

Conclusión

Este proyecto demuestra la funcionalidad básica de una aplicación bare-metal ARM, incluyendo comunicación UART de bajo nivel, gestión de memoria y manipulación de cadenas. Al implementar esta calculadora simple, se puede obtener experiencia práctica en programación ARM, desarrollo en C y sistemas embebidos.